Keeping Hub Links: Pairwise Evidence for Fraud Ring Detection on Streaming Transaction Graphs

Roshan Parveen
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
roshan.parveen2023@vitstudent.ac.in

Sanjay Kumar S
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
sanjaykumar.s2023@vitstudent.ac.in

***Abstract*—Rule-based fraud-ring detectors hide busy shared attributes, such as office gateways, behind a degree cap. We study a streaming pipeline that keeps these links as pairwise evidence, detects incrementally and scores candidate rings. On a simulator with ring-level ground truth, using protocols committed before each run and fresh test seeds, a cap inside the pipeline costs 0.08 ring F1. Even a cap that removes only attributes unable to link a pair alone costs 0.051 (exploratory). Adding a graph neural network as a detector, with a ring scorer that reads its member probabilities, outperforms the strongest existing method, a GraphSAGE classifier, by 0.102 F1 (95% CI +0.067 to +0.138); the result replicates on two further blocks. Ablations locate the gain in this scorer, not in gating message passing around busy attributes. The advantage needs training-like inputs: when benign and ring transfer amounts overlap, the GNN-inclusive pipelines lose precision and the rule-based pipeline is more robust. All data are synthetic.**

***Index Terms*—fraud rings, property graphs, streaming graph analytics, incremental detection, pairwise evidence, graph neural networks**

I. Introduction

Fraud rings are groups of accounts that act together: they share devices, payment cards or network addresses, and they move money among themselves in cycles or fan-in and fan-out patterns. Graphs make these relations explicit, which is why graph-based methods dominate fraud and anti-money laundering (AML) research [1], [2]. Two problems appear as soon as such methods run on a live stream of events.

The hub dilemma. Legitimate shared infrastructure, such as an office gateway, a carrier-grade network address translation (NAT) address or a store kiosk, links many unrelated accounts. Linking accounts by shared properties produces sparse links through strongly identifying entities and over-linkage through widely shared ones such as IP addresses [3]. The usual remedy in rule-based systems is a hard degree cap: attributes shared by more than c accounts are ignored. The cap removes the noise but also removes the evidence that a group shares a busy attribute, which opens an evasion path, much as fraudsters elsewhere hide by connecting to benign entities [4]. Keeping such links is not enough either: if one busy attribute can link two accounts on its own, every co-user of the hub is linked.

Latency on a stream. Accounts, links and transfers arrive continuously. Re-running whole-graph queries on a timer delays detection, and the cost of each pass grows with the graph rather than with the change, a problem that incremental community detection addresses for clustering [5], [6].

We make two contributions and support them with a system and an evaluation.

- **Contribution: keep hub links, but never let a hub link a pair alone.** Pairs of accounts accumulate decayed evidence from shared attributes and from transfers between them, and a pair is linked only when a busy attribute is joined by a second piece of evidence. Isolation experiments on fresh test seeds show that this condition, not the shape of the degree discount, is what matters: within the same pipeline a degree cap costs 0.08 ring F1 (95% CI 0.05 to 0.12) and most hub-hiding rings (9/52 against 29/52), a weight that lets one attribute suffice costs 0.14 (0.44 on the hub-hiding scenario), and two different weights that satisfy the condition are equivalent within 0.05 F1 (-0.008, CI -0.020 to +0.003), though not every such weight is (a step weight costs 0.041; Section V-C). The degree discount we use, $1/\ln(1+d)$, is a variant of FRAUDAR's column weighting $1/\log(d+c)$ [7] (with $c = 5$ in its experiments); we do not claim it as new. Even a cap that removes only attributes unable to link a pair alone costs 0.051: busy attributes help as supporting evidence.
- **Contribution: a GNN-inclusive pipeline that beats the existing methods.** The pipeline with a GNN as an extra detector, and a ring scorer that reads the GNN's member probabilities (trained with out-of-fold scores), outperforms the strongest existing method, a GraphSAGE classifier, by +0.102 F1 (95% CI +0.067 to +0.138, confirmatory), and the gain replicates on two further fresh blocks. Ablations show that the GNN-aware scorer carries the gain, while gating the GNN's message passing around busy attributes, which we designed for this, hurts. A plain GraphSAGE inside the pipeline does as well as our gated model. The advantage depends on training-like transfer amounts (Section V-D).
- **System.** A micro-batch engine re-evaluates only the region each batch touches, with fan-out limits; we state when it misses no pair that a full pass would link, and show that ring-level equality with full recomputation

holds for small hubs only (Section III). Candidate rings are scored by a calibrated classifier, and alerts carry feature contributions, an evidence subgraph and, where one exists, a counterfactual.

- **Evaluation.** A deterministic simulator with ring-level ground truth and five evasion strategies; protocols committed before the runs, with fresh test seeds for every new experiment; and a comparison with six existing methods (tuned rules, Louvain, Leiden, FRAUDAR-style peeling and a GraphSAGE classifier), each with a scorer trained on its own candidates and at least the same tuning budget (Section V-B).

II. Related Work

Graph fraud detection. Most recent graph fraud detection work trains graph neural networks (GNNs) to label nodes. It addresses heterophily, where fraudulent nodes connect mostly to benign ones [8], [9]; camouflage, by reweighting neighbours adversarially [4]; scarce or unreliable labels, with one-class and semi-supervised training [10]–[12]; and interpretability [13], usually on GraphSAGE-style neighbourhood aggregation [14]. These methods output per-node scores and need labelled training graphs; we compare with a GraphSAGE classifier and a fixed rule that turns flagged nodes into rings (Section V-B). GB-FRD [2] targets fraud rings in peer-to-peer payment graphs, as we do, and Yin et al. [3] replace hard shared-property links with behavioural similarity because of over-linkage through shared entities. We keep shared-attribute links, which are cheap and explainable, and change how much each one counts.

Dense subgraphs and peeling. Finding unusually dense groups is a classical route to fraud and collusion detection: greedy peeling approximates the densest subgraph [15], spectral and greedy densest-subgraph algorithms scale to billion-edge graphs [16], and sequential peeling of a weighted bipartite graph exposes coordinated adversarial annotators [17]. FRAUDAR [7] makes peeling resistant to camouflage by weighting each edge by a decreasing function of the degree of the object it points to, recommending $1/\log(x+c)$ and arguing that $1/x$ shrinks too fast; Spade [18] maintains such peeling-based dense subgraphs incrementally under edge insertions on transaction graphs, with FRAUDAR's weighting and the same peeling sequence as a full recomputation. Core decomposition, the peeling primitive we use to extract dense cores, has efficient parallel and GPU implementations [19], [20], and greedy peeling by minimum degree is the classical approximation of the densest subgraph [15]. Our degree discount is therefore not new: it is a variant of FRAUDAR's column weighting (which uses $c = 5$ rather than our $c = 1$), and close to Adamic–Adar. What we study is different from Spade's setting: pairwise evidence between accounts from shared attributes and internal transfers, and the condition under which a busy shared attribute helps rather than hides a ring. We run FRAUDAR-style peeling, whose output Spade maintains incrementally, as a baseline.

Community detection on dynamic graphs. Louvain modularity optimisation [21] remains a standard way to find groups, and Leiden [22] refines it so that communities are guaranteed to be connected; we use both as baselines. Incremental community detection avoids repartitioning a changing graph by updating only the affected communities [5], [6]. Community structure, however, is not ring structure: a benign hub joins thousands of users into one community.

Similarity weighting. Local similarity indices such as Jaccard and Adamic–Adar [23], which discounts a shared neighbour by the logarithm of its degree, are standard in link prediction [24]. Our hub weight applies a similar discount to shared attributes, and we match flagged rings to true ones by Jaccard overlap. Entity resolution faces the same choice between discounting and capping: meta-blocking weights candidate pairs by the inverse size of the blocks they share, and block purging drops oversized blocks [25].

AML on public data. Work on the Elliptic Bitcoin graph compares tabular ensembles with GNNs under temporal splits [26]–[29]; AMLworld provides synthetic laundering attempts with typology labels [30]. These studies classify transactions, so their scores are not comparable with our ring-level metrics (Section V-I).

Calibration and explanations. Post-hoc calibration by isotonic regression [31] and its comparison with Platt scaling [32] make classifier scores usable as probabilities; Azbergen et al. [33] compare such calibrators for AML risk scores, and input-dependent (local) temperature scaling extends calibration to per-pixel predictions [34]. Counterfactual explanations state what would have to change for a different decision; their robustness [35], consistency across model updates [36] and per-feature importance [37] are active topics. We attach simple, model-specific counterfactuals to every alert.

III. Method

Fig. 1 shows the system. A seeded simulator emits observable events (account joins and leaves, attribute links, transfers); ground truth goes only to a ledger the detector never sees.

A. Graph model

Users link to devices, cards and IP addresses (`USED_DEVICE`, `HAS_CARD`, `LOGGED_IN_FROM`) and send transfers to each other. The same contract is implemented over Neo4j (Cypher) and over an in-memory store with identical semantics; all experiments below use the in-memory store. In live runs, events reach the engine through an in-process queue or Redis Streams, whose consumer settings (batch size, blocking timeout) trade throughput against latency [38].

B. Pairwise evidence and hub weighting

For two users u, v the edge weight is

$$W(u,v) = \sum_{a \in A(u) \cap A(v)} h(\deg a)\, \delta(t_a) + \min\Big(\beta \sum_{x \in T(u,v)} \delta(t_x),\, \beta_{\max}\Big), \tag{1}$$

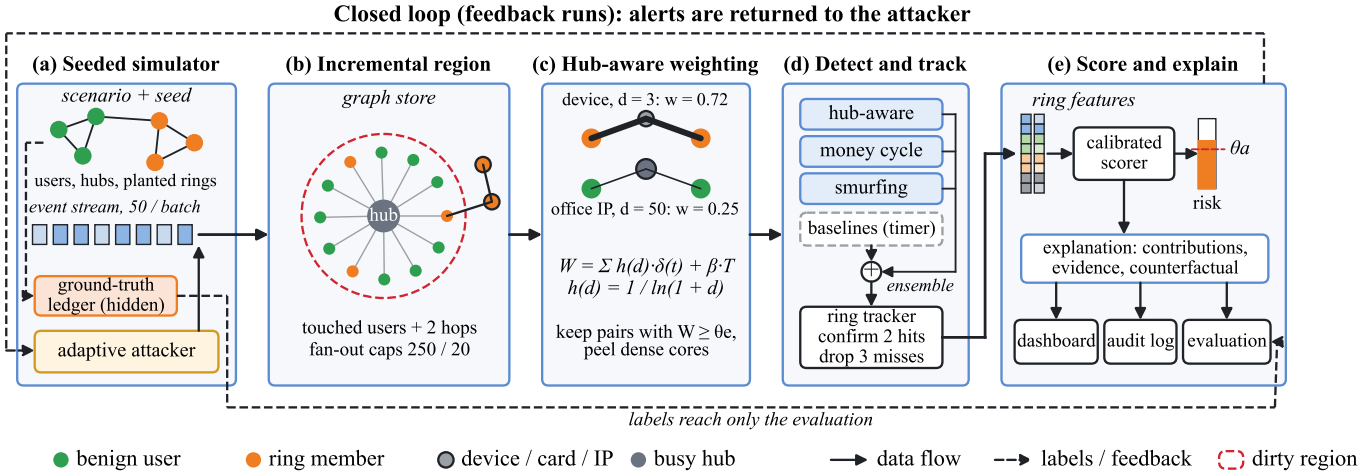


Fig. 1. Overview of the system. (a) A seeded simulator emits the observable event stream in micro-batches; labels go only to a ledger. (b) Each batch is written to the graph store and expanded to the bounded region it touched. (c) Shared attributes are weighted by their degree, so a busy hub counts for little and a rarely shared device for much. (d) Incremental detectors run on the region and a tracker confirms rings across batches. (e) Confirmed rings are scored, explained and alerted. Solid arrows: data flow; dashed arrows: labels (evaluation only) and, in closed-loop runs, alerts returned to the attacker.

where $h(d) = 1/\ln(1+d)$, $\delta(t) = 2^{-t/\tau}$ is exponential decay with half-life $\tau = 300\,\text{s}$, t_a is the age of the older of the two links to attribute a, and $T(u, v)$ are the transfers between u and v in the window, with $\beta = 0.15$ and $\beta_{\max} = 0.6$. A device shared by three accounts weighs 0.72; an office address shared by fifty weighs 0.25. At $\theta_e = 0.3$ an attribute shared by more than about 27 accounts cannot link a pair on its own: it needs a second shared attribute or transfers between the two accounts. A rarely shared attribute can ($h(2) \approx 0.91$). Section V-C tests which part of this rule matters. Candidate rings are the connected components of the graph of pairs with $W \geq \theta_e$ (and a non-zero attribute term), peeled to cores of edge density at least 0.75 and size 3–25. The edge threshold θ_e is selected on validation seeds (Section IV-B).

C. Incremental detection

Events are grouped into micro-batches of 50. For each batch the engine takes the users it touched, expands them by $k = 2$ hops through shared attributes and transfers, and builds a view of that region over a $900\,\text{s}$ window, together with the *global* degree of every attribute in it, so $h(\deg a)$ is the same as in a full pass. The first hop may cross attributes with up to 250 users (so the co-members of a hub are re-examined); later hops cross only attributes with at most 20 users, which stops the region from spreading through hubs. The first-hop limit is itself a degree cap on the region. Transfers are always crossed, so a touched account u and a partner v can end up in different regions only if every attribute they share has more than 250 users and they have no transfer in the window; their weight is then at most $k\,h(251)$ for k such attributes. With $h(d) = 1/\ln(1+d)$ and $\theta_e = 0.3$ (and equally for a constant weight at $\theta_e = 1.0$, since a single link decays below 1) a missed pair needs $k \geq 2$. A variant that crosses an over-limit attribute whenever the shared over-limit weight could reach θ_e closes this gap exactly; in our runs it never added a user. Equality with full recomputation is a property of pairs, not of rings: candidate rings come from greedy peeling of connected components, which is not local, so at hubs of 500 or more accounts individual rings differed between the two modes in both directions (Section V). Incremental detectors (hub-aware shared attribute, temporal cycle, smurfing) run on the view; whole-graph detectors (the static Cypher rules and community detection) run on a timer.

D. Ensemble, tracking and scoring

Candidates from different detectors are merged when their Jaccard overlap is at least 0.5, or when one contains 80% of the other and they differ in size by at most a factor of two. A tracker matches candidates to existing rings across batches; a ring is *confirmed* after two consecutive hits and dissolved after three misses (this hysteresis is what the ablation "w/o hysteresis" sets to one hit and one miss). Confirmed rings are scored by a logistic-regression model over 19 ring features (size, density, which attribute kinds are shared, the hub-adjusted weight, cycle presence and length, burstiness, account ages, internal transfers per member, amount statistics, growth, hub exposure and the number of agreeing detectors), calibrated with isotonic regression [31]. A ring becomes an alert when its calibrated risk reaches the alert threshold θ_a. Each alert carries the per-feature contributions to the score, the subgraph of shared attributes and transfers that formed it, and, where one exists, a counterfactual (for the leading features, the value at which the risk would fall below the threshold, and for hub-aware rings the degree the shared attribute would need before the rule stops firing). The model is trained on seeds disjoint from all evaluation seeds (Fig. 2): the detector runs on six scenarios with seeds 101–110, each confirmed or changed ring becomes a row of its 19 features, labelled from the hidden ledger, and within each run the first 70% of rows fit the regression and the last 30% the calibrator. On the scorer hold-

TABLE I
SCORER TRAINED ON ALL SIX SCENARIOS VS. TRAINED WITHOUT THE SCENARIO IT IS TESTED ON (SCORER HOLD-OUT SEEDS 201–205).

Held-out scenario	Snapshot ROC AUC all	Snapshot ROC AUC left out	Ring PR AUC all	Ring PR AUC left out
adversarial	0.922	0.912	0.550	0.534
baseline	0.988	0.987	1.000	1.000
drift	0.995	0.994	0.856	0.763
high_noise	0.981	0.978	0.888	0.912
hub_heavy	0.995	0.992	0.938	0.872
scale_sweep	0.985	0.982	0.914	0.868

TABLE II
SCENARIOS (FROM `configs/scenarios/*.yaml`).

Scenario	Users	Dur. (s)	Ev./s	Rings and evasion
baseline	30	60	10	4 planted, none
high_noise	150	60	50	4 planted + random, decoys
hub_heavy	100	60	25	4 planted, hub hiding
adversarial	120	90	30	5 planted + random, all five
drift	200	90	40	2 planted + random, hub hiding
demo_story	120	90	20	3 planted, hub hiding
scale_sweep	10^3+	120	100	latency benchmark (RQ2)

out seeds 201–205 (six scenarios) it reaches ROC AUC 0.982 and PR AUC 0.849 over 1,961 ring units (a unit is one tracked ring in one run, scored by its highest calibrated score); over the 4,516 correlated snapshots of those rings the figures are 0.982 and 0.908. Trained without one scenario and tested on it, the scorer loses at most 0.01 ROC AUC (`adversarial`) and at most 0.09 ring-level PR AUC (`drift`; Table I), but all scenarios come from the same generator.

IV. EXPERIMENTAL SETUP

A. Simulator and scenarios

The simulator is a deterministic discrete-event generator: a scenario file and a seed fix the whole stream, which we verify by a digest of the event sequence recorded with every result. Benign users join and leave, link to devices, cards and IPs drawn from heavy-tailed popularity, and transact; benign hubs (offices, carrier NAT, kiosks, family devices) are generated with configured sizes. Ring types are shared device, card or IP, mixed attribute, money cycle, layered chain and smurfing fan-in/fan-out. Evasion strategies are hub hiding (the ring shares a busy benign hub), low-and-slow activity, identity rotation, decoy edges to benign users, and an adaptive attacker that shifts its strategy mix away from strategies that were flagged. Fig. 3 shows the generator and its two outputs, and Table II lists the scenarios.

B. Protocol

Seeds are split by purpose: scorer training (101–110) and scorer hold-out (201–205), threshold selection (validation, 301–303), and evaluation. The original evaluation (Tables III–IX) uses test seeds 1001 and up. Every experiment added in revision uses a block of fresh test seeds declared, with its hypotheses, in a protocol committed before its first run (`docs/research/`): 2001–2010 for the degree-cap control, 2101–2120 for the comparison with existing methods, 2301–2320 for the hybrid, 2401–2420 for the evidence condition and 2501–2505 for scale. The weight-function comparison (1001–1030), the hub-size sweep (1001–1005) and the timer comparison (1001–1005) reuse the original test seeds and are exploratory. Round-three experiments use 3001–3720 (`docs/research/round3_protocol.md`). Each block is used once. On the validation seeds we evaluated a grid of alert thresholds $\theta_a \in \{0.2, 0.3, 0.5, 0.7, 0.8, 0.9\}$ and edge thresholds $\theta_e \in \{0.2, 0.3, 0.4, 0.5, 0.6\}$ on the five accuracy scenarios; the grid was widened whenever the best value sat on its edge. The selected values are $\theta_a = 0.8$ and $\theta_e = 0.3$ (mean validation F1 0.72). We report the proposed method both with these values and with the a-priori values $\theta_a = \theta_e = 0.5$.

Every method in an experiment processes the *identical* event stream for a given scenario and seed (the attacker does not receive feedback in these runs), so differences between methods are paired. In the original tables we report the mean and 95% t-interval over runs and test differences with the Wilcoxon signed-rank test on paired runs (`zsplit` handling of ties), with Cohen's d_z as effect size. Because the paired difference depends on the scenario, the revision experiments report the scenario-weighted mean difference with a 95% cluster bootstrap interval (seeds resampled within scenario, 10,000 resamples), a scenario-stratified sign-flip permutation test, per-scenario differences, and win/tie/loss counts. Four comparisons are confirmatory and form one family with Holm correction: the degree-cap control, the comparison with the strongest existing method, the hybrid against that method, and the evidence condition. Their raw (Holm-adjusted) sign-flip p-values are 0.002 (0.007), 0.025 (0.076), 0.664 (1.00) and 0.980 (1.00). The family was fixed in the protocols before the runs; all other tests are exploratory and uncorrected. We call two variants equivalent only when the 90% interval of their difference lies within ± 0.05 F1 (about one ring in every two to four runs), and we report results on five or fewer seeds as ranges, without p-values.

C. Metrics

A flagged ring *matches* a true ring when their member sets overlap with Jaccard at least 0.5. Ring precision is the fraction of flagged rings that match a true ring; ring recall is the fraction of true rings matched at some point while active. A run in which nothing is flagged has undefined precision; we count its F1 as 0. Time to detect (TTD) is the time from a ring's activation to its first matching flag. Detection latency is the wall-clock time per micro-batch for the dirty-region expansion, the view and the incremental detectors.

D. Baselines

- **Static Cypher, cap 100 / cap 10.** The original rules: shared device (Q1), shared card (Q2), 3-cycles of transfers (Q3), and shared IP with a degree cap (Q4a/Q4b), run on the whole graph every 5 s of simulated time.

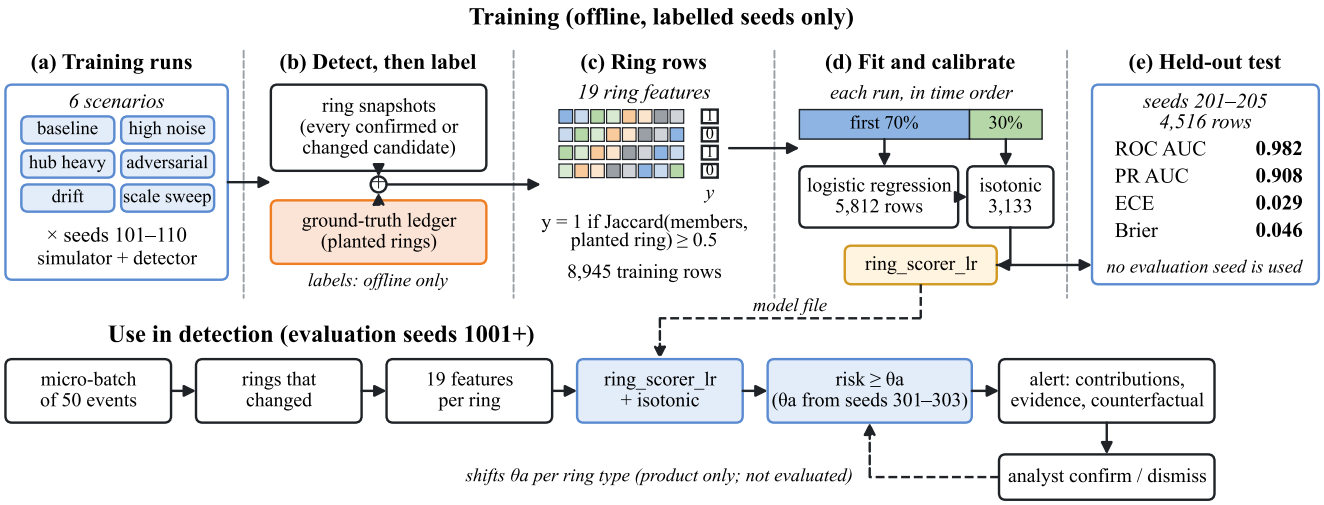


Fig. 2. Training and use of the ring scorer. (a)–(b) The simulator and detector run on training seeds, and ring snapshots are labelled from the ground-truth ledger, which the detector never reads. (c) Each snapshot is a row of 19 ring features. (d) Within each run, rows in time order are split 70/30 between logistic regression and isotonic calibration. (e) Scorer hold-out seeds 201–205: snapshot ROC AUC 0.982, PR AUC 0.908; ring level 0.982 and 0.849 (values from `models/ring_scorer_lr.json` and `results/e6b_generalisation`). Below: in detection, each changed ring is scored every batch, and an alert carries its explanation; analyst feedback adjusts the threshold in the product; that loop is not part of any experiment in this paper.

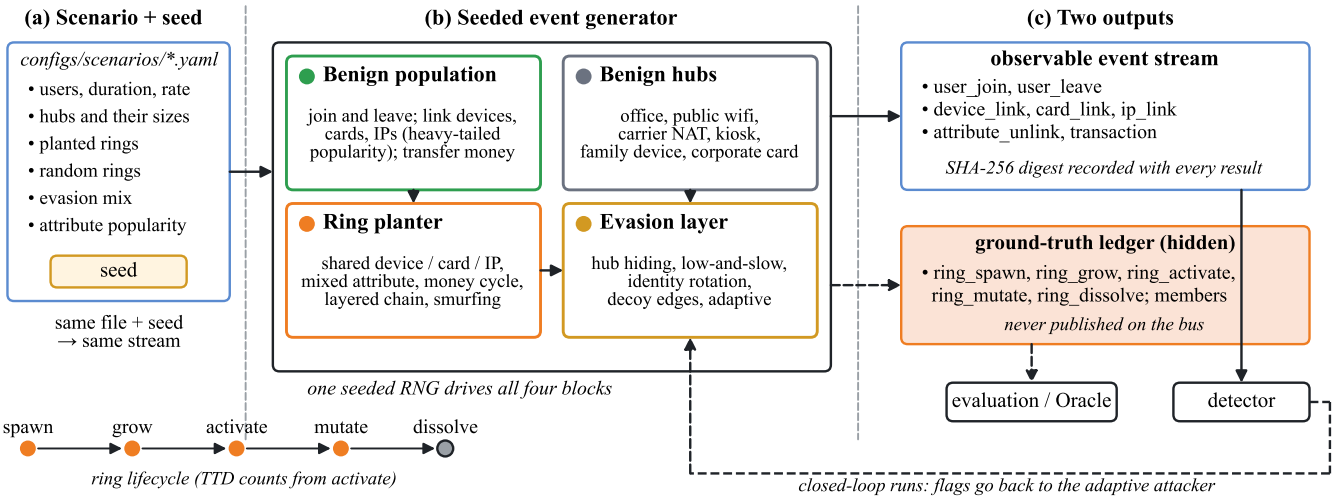


Fig. 3. Simulator design. (a) A scenario file and a seed fix the whole run. (b) One seeded generator combines a benign population, benign hubs, planted rings and an evasion layer. (c) Observable events go to the detector, with a digest of the sequence recorded with every result; ring lifecycle events go only to the ground-truth ledger used for evaluation and for the demo's Oracle view, which overlays that ground truth on the dashboard and is not shown to the detector. In closed-loop runs, flags are returned to the adaptive attacker.

- **Louvain.** Louvain communities [21] of the user graph projected through shared attributes and transfers, kept when dense enough.
- **Connected components.** Weakly connected components of the same projection, no density filter.

All baselines use the same two-hit confirmation as the proposed method. Section V-B adds six existing methods under a common protocol (gradient boosting, the learned baseline of Section VI-D, is the sixth).

Fair comparison. All methods process identical event streams, emit candidate rings of 3–25 accounts, pass them through the same tracker and are matched to true rings by Jaccard overlap of at least 0.5. Each method is scored by a logistic-regression model with isotonic calibration trained on that method's own candidates (seeds 101–110, the protocol of Section III). Each method's structural parameter (degree cap and cycle rule for the rules; resolution for Louvain and Leiden; minimum block score for peeling; node threshold for GraphSAGE) and its alert threshold are selected jointly on the validation seeds over at least as many grid points as

the proposed method received (60 to 84 against 30). Peeling, Leiden and GraphSAGE recompute on the whole graph after every micro-batch; the rules and Louvain keep their 5 s timer.

E. Environment

All runs used one laptop (Intel Core i3-1115G4, 2 cores / 4 threads, 7.7 GB RAM, Windows 11), Python 3.11.9, NumPy 2.2.6, scikit-learn 1.7.2, NetworkX 3.5, with experiments run one at a time. The revision experiments ran on a 2-core Linux container with the same library versions (pinned in `requirements-lock.txt`; PyTorch 2.5.1 and leidenalg 0.12 for the new baselines), several at a time, so we report no latencies from them. With newer library versions the event streams are identical but detection results can differ, so the lock file is part of the result.

V. Results

A. RQ1: accuracy against baselines

Table III pools 5 scenarios $\times$ 10 test seeds for every method, all on identical streams. Without a scorer, the static rules reach ring F1 0.21 with a degree cap of 10 and 0.19 with a cap of 100, Louvain 0.29 and connected components 0.00. These rows differ from the proposed method in two ways at once: how candidates are generated and whether they are scored. With the same calibrated scorer and an alert threshold selected per method on the validation seeds, the static rules reach 0.56 (cap 10) and 0.52 (cap 100) and Louvain 0.41, against 0.73 ($\pm$0.05) for the proposed method; every difference to the proposed method remains significant after Holm correction ($p < 0.001$). Within the scored group the differences isolate the candidate generator as a whole (Section V-C separates its parts); between a generator's unscored and scored rows they isolate the scorer. The shipped scorer was trained on the proposed pipeline's candidates, which favours it; Section V-B gives every method its own scorer. For the rules with cap 10, the scorer raises F1 from 0.21 to 0.56, through precision (0.14 to 0.64) at some cost in recall (0.61 to 0.53); the candidate generator raises it further to 0.73, mostly through recall (0.53 to 0.74). The scorer was trained on candidates of the proposed pipeline, so on rule and Louvain candidates it works under some distribution shift. The unscored static rules reach recall 0.61 but precision only 0.14: the shared-attribute rules and the 3-cycle rule fire on many benign groups, and two-hit confirmation does not remove them. Connected components never produce a group within the ring size limits, because benign hubs join almost all users into one component; Louvain splits that component, but its communities rarely coincide with a ring. The thresholds selected on validation seeds improve on the a-priori ones (F1 0.73 vs. 0.68, $p = 0.034$, Holm over the eight comparisons of Table III).

Per scenario (Table IV), the gap to the static rules is largest on `drift` and `high_noise`, where many benign groups satisfy the rules and their precision collapses, and smallest on the clean `baseline` scenario and on `adversarial`, which is the hardest scenario for every method. Against the scored rules the difference depends on the scenario: +0.34 on `high_noise`, but only +0.03 (95% CI -0.11 to +0.17, wins/ties/losses 5/0/5) on `hub_heavy`, the scenario built for hub hiding. With their own scorer the rules reach 0.63 on these streams.

Degree cap vs. hub weighting. Fig. 4 isolates the mechanism this paper replaces. Only shared-attribute and mixed rings count as ground truth here (flags that match only other ring types are ignored), the static rules run without the cycle rule, and the hub-aware detector runs alone. Without the scorer, the rules' recall is 0.55 for every cap from 5 to 200, although caps of 50 and more exceed every benign hub in these scenarios: the rules miss these rings because each rule returns all users of one attribute, so a ring inside a larger group fails the Jaccard match, whatever the cap. Section V-C tests the cap where it does act, inside the pairwise detector. Hub weighting reaches recall 0.86, but on its own at precision 0.15. With the calibrated scorer on top, hub weighting reaches recall 0.81 (95% bootstrap interval 0.70–0.91; precision 0.35) and its best F1 is 0.58 (alert threshold 0.8); the cap with the scorer stays below recall 0.53 (0.48–0.58) and its best F1 is 0.55. The difference in best F1, 0.03, has a 95% bootstrap interval of -0.03 to 0.10 over the 20 runs (the best threshold re-selected in every resample; whole runs resampled within scenario), so at the best operating point the two mechanisms are not distinguishable on F1. The best operating point is chosen on these test runs for both arms, so both best-F1 values are optimistic. At low recall the cap with the scorer is the more precise of the two (0.72 at recall 0.45). Hub weighting therefore does not dominate the cap everywhere; it extends the frontier to recall levels the cap cannot reach, and the scorer is what makes those levels usable.

B. Comparison with existing methods

Table V compares the pipeline with six existing methods under the protocol of Section IV, on test seeds no experiment had used. Against every non-learned method it is ahead: by 0.059 ring F1 (95% CI +0.023 to +0.094) against the rules with a cap and cycle rule tuned per scenario, which is the strongest rule baseline and uses the scenario's identity, by 0.072 against the rules tuned globally, and by 0.198, 0.212 and 0.263 against Louvain, Leiden and FRAUDAR-style peeling. With their own scorers the rules come much closer than in Table III (F1 0.646 against 0.56 with the shipped scorer). Peeling and Leiden are precise but find few rings, and peeling and Leiden find almost no decoy rings (2/59 and 4/59).

A GraphSAGE classifier trained on ring-membership labels is the one method ahead of the pipeline: F1 0.778 against 0.719 (difference -0.059, 95% CI -0.097 to -0.023, confirmatory). With a ring scorer of its own GraphSAGE reaches 0.751, and the difference (-0.033) is not significant. The gap is not uniform: GraphSAGE is ahead on `drift` (-0.37) and `high_noise` (-0.16), behind on `adversarial` (+0.10), and the pipeline finds more adaptive rings (24/28 against 18/28) and decoy rings (50/59 against 37/59). In a separate decoy sweep on `high_noise` (seeds 3301–3310, exploratory), GraphSAGE finds more decoy rings at the default strength. At eight decoys per action its decoy-ring recall falls to 16/29,

TABLE III

RING-LEVEL DETECTION ON 5 SCENARIOS × 10 TEST SEEDS, POOLED (MEAN ± 95% CI OVER RUNS; A RUN WITH NO FLAGS HAS UNDEFINED PRECISION AND COUNTS AS F1 = 0). TTD: MEDIAN TIME FROM RING ACTIVATION TO FIRST FLAG. LATENCY: MEDIAN PER-BATCH DETECTION TIME ON THE IN-MEMORY STORE; †FULL-GRAPH METHODS RUN ON A 5 S TIMER, NOT PER BATCH, SO THEIR ENTRY IS THE MEDIAN TIME OF ONE FULL-GRAPH PASS. p: PAIRED WILCOXON SIGNED-RANK TEST OF F1 AGAINST THE PROPOSED METHOD ON IDENTICAL STREAMS. SCORED BASELINES USE THE SHIPPED SCORER (`MODELS/RING_SCORER_LR.JSON`), WITH THE ALERT THRESHOLD SELECTED PER METHOD ON THE VALIDATION SEEDS; THE PROPOSED METHOD INCLUDES THE SAME SCORER, SO ONLY ROWS WITHIN A GROUP ISOLATE THE CANDIDATE GENERATOR. p VALUES ARE HOLM-CORRECTED OVER THE EIGHT COMPARISONS OF THIS TABLE. SECTION V-B GIVES EVERY METHOD ITS OWN SCORER.

Method	Precision	Recall	F1	TTD (s)	Latency (ms)	p
Without a scorer						
Static Cypher (cap 100)	0.12 ± 0.03	0.61 ± 0.08	0.19 ± 0.05	9.5	6.0†	<0.001
Static Cypher (cap 10)	0.14 ± 0.04	0.61 ± 0.08	0.21 ± 0.05	9.5	5.9†	<0.001
Louvain communities	0.39 ± 0.09	0.33 ± 0.08	0.29 ± 0.06	18.8	38.4†	<0.001
Connected components	–	0.00 ± 0.00	0.00 ± 0.00	–	11.1†	<0.001
With the calibrated scorer						
Static Cypher (cap 100) + scorer	0.63 ± 0.08	0.48 ± 0.09	0.52 ± 0.08	8.6	4.5†	<0.001
Static Cypher (cap 10) + scorer	0.64 ± 0.07	0.53 ± 0.09	0.56 ± 0.07	9.3	4.6†	<0.001
Louvain communities + scorer	0.81 ± 0.12	0.32 ± 0.09	0.41 ± 0.09	19.0	27.9†	<0.001
Hub-aware incremental (proposed)	0.76 ± 0.06	0.74 ± 0.06	**0.73 ± 0.05**	9.3	26.6	–
Proposed, a-priori thresholds	0.73 ± 0.06	0.69 ± 0.06	0.68 ± 0.05	9.0	22.7	0.034

TABLE IV

RING F1 PER SCENARIO (MEAN ± 95% CI OVER 10 TEST SEEDS).

Method	baseline	high_noise	hub_heavy	adversarial	drift
Static Cypher (cap 100)	0.48 ± 0.07	0.06 ± 0.02	0.24 ± 0.04	0.14 ± 0.04	0.05 ± 0.02
Static Cypher (cap 10)	0.54 ± 0.08	0.07 ± 0.02	0.27 ± 0.05	0.14 ± 0.04	0.05 ± 0.02
Louvain communities	0.44 ± 0.09	0.26 ± 0.19	0.24 ± 0.09	0.14 ± 0.09	0.38 ± 0.16
Connected components	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Hub-aware incremental (proposed)	0.99 ± 0.03	0.62 ± 0.08	0.79 ± 0.08	0.59 ± 0.14	0.67 ± 0.04
Proposed, a-priori thresholds	0.93 ± 0.07	0.66 ± 0.05	0.66 ± 0.12	0.53 ± 0.11	0.62 ± 0.09

while the pipeline's stays at 20–23 of about 28. Ranked by their calibrated scores, the pipeline's rings have the highest ring-level PR AUC of all scored methods (0.780 against 0.719 for GraphSAGE with a scorer; difference +0.061, 95% CI +0.007 to +0.116, exploratory), so GraphSAGE's advantage lies in its operating point, not in ranking. All of these data come from one generator, and GraphSAGE learns its regularities from about 320,000 labelled user snapshots.

Hybrid. Adding the GraphSAGE classifier as a fourth detector inside the pipeline, with a ring scorer trained on the combined candidates, was decided after this comparison, so we tested it on another fresh block (2301–2320, bottom of Table V). The hybrid reaches F1 0.770 against 0.780 for GraphSAGE alone (hybrid minus GraphSAGE −0.010, 95% CI −0.044 to +0.023) and 0.718 for the pipeline alone. At the same F1 it finds many more hub-hiding rings than GraphSAGE alone (89/107 against 68/107), at lower precision (0.76 against 0.88). Any gain of the hybrid belongs to the combination, not to a single element of the pipeline. At matched recall of about 0.67 the pipeline, GraphSAGE and GraphSAGE with a scorer have the same precision (0.86–0.89, seeds 3401–3405, descriptive). GraphSAGE reaches no recall above 0.69 at any threshold, while the pipeline and the hybrid reach 0.78–0.80.

C. What the advantage depends on

Table VI isolates the parts of the pairwise rule inside the full pipeline, on fresh seeds. Removing hub links with a degree cap of 10 costs 0.08 F1 (95% CI 0.05 to 0.12, confirmatory, $p = 0.002$), 0.34 on `hub_heavy`, where recall of hub-hiding rings falls from 29/52 to 9/52. Keeping hub links but letting a single attribute link a pair (a constant weight with $\theta_e = 0.5$) costs 0.14, and 0.44 on `hub_heavy`: every co-user of the hub is linked and the ring cannot be separated from them. Under the condition that a busy attribute cannot suffice alone, the weight's shape does not matter: $1/\ln(1+d)$ and a constant weight differ by −0.03 (CI −0.07 to +0.01) without a cap, and by −0.008 (CI −0.020 to +0.003) when both also bar rare attributes from linking alone. That last restriction (the confirmatory test of the evidence condition) changes nothing overall (0.000, CI −0.028 to +0.028) but trades scenarios: it removes almost all false rings on `drift` (1.8 to 0.05 per run; F1 +0.15) and loses adaptive rings (22/32 to 10/32; `adversarial` −0.14), which often link through one rarely shared device. Finally, with the same cap the pairwise pipeline and the scored rules tie overall (+0.005) but trade sharply: the pipeline wins under benign noise (+0.28 on `high_noise`) and the rules win on hub hiding (−0.30 on `hub_heavy`). The capped rules do not find these rings through the hub. On `hub_heavy` (seeds 3001–3020) they detect 41 of 60 hub-hiding rings: 27 through the 3-cycle transfer rule and 14 through a device or card that only the ring shares. A cap therefore hides rings from a detector that relies on the hub link, the pairwise rule, and not from rules that use other evidence the

TABLE V

COMPARISON WITH EXISTING METHODS ON FRESH TEST SEEDS 2101–2120 (5 SCENARIOS × 20 SEEDS, PAIRED). EVERY METHOD HAS A SCORER TRAINED ON ITS OWN CANDIDATES (SEEDS 101–110) UNLESS NOTED, AND ITS STRUCTURAL PARAMETER AND ALERT THRESHOLD SELECTED ON SEEDS 301–303 WITH AT LEAST THE PROPOSED METHOD'S BUDGET. PR AUC: RING-LEVEL, THRESHOLD-FREE (NOT DEFINED WITHOUT A RING SCORE). Δ: PROPOSED MINUS ROW (SCENARIO-WEIGHTED, 95% CLUSTER BOOTSTRAP), p: STRATIFIED SIGN-FLIP; THE COMPARISON WITH THE STRONGEST BASELINE BY VALIDATION F1 (GRAPHSAGE, NO SCORER) IS CONFIRMATORY, THE OTHERS EXPLORATORY. ‡EXPLORATORY VARIANT ADDED AFTER THE DEGREE-CAP CONTROL: THE PIPELINE WITH $h(d) = 1$ FOR EVERY ATTRIBUTE AND $\theta_e = 1.0$ (SELECTED ON VALIDATION), SO NO SINGLE ATTRIBUTE LINKS A PAIR; SAME SCORER AND θ_a AS THE PROPOSED METHOD. §SELECTION USES THE SCENARIO'S IDENTITY (AN ORACLE ADVANTAGE).

Method	P	R	F1	PR AUC	Hub-hiding	Decoy	Adaptive	Δ	95% CI	p
Proposed (shipped)	0.74	0.72	0.719 ± 0.040	0.780	65/105	50/59	24/28	–		
Proposed, constant weight‡	0.81	0.71	0.744 ± 0.043	0.755	69/105	50/59	11/28	−0.025	[−0.054, +0.003]	0.192
GraphSAGE, no ring scorer	0.88	0.72	0.778 ± 0.040	–	73/105	37/59	18/28	−0.059	[−0.097, −0.023]	0.025
GraphSAGE + scorer	0.85	0.69	0.751 ± 0.041	0.719	69/105	37/59	16/28	−0.033	[−0.072, +0.007]	0.227
Rules, cap tuned per scenario§	0.87	0.58	0.659 ± 0.053	0.619	71/105	23/59	22/28	+0.059	[+0.023, +0.094]	0.014
Rules, cap and cycle rule tuned	0.91	0.55	0.646 ± 0.055	0.655	54/105	23/59	18/28	+0.072	[+0.037, +0.106]	0.003
Louvain, resolution tuned	0.61	0.48	0.521 ± 0.051	0.562	43/105	28/59	11/28	+0.198	[+0.158, +0.239]	<0.001
Leiden, resolution tuned	0.86	0.40	0.507 ± 0.054	0.678	5/105	4/59	4/28	+0.212	[+0.172, +0.252]	<0.001
FRAUDAR-style peeling (= Spade output)	0.85	0.37	0.456 ± 0.058	0.519	58/105	2/59	5/28	+0.263	[+0.221, +0.304]	<0.001
Gradient boosting on Louvain groups	0.91	0.33	0.427 ± 0.062	0.510	0/105	7/59	3/28	+0.291	[+0.249, +0.334]	<0.001
Hybrid, fresh seeds 2301–2320 (Δ: hybrid minus row)										
Proposed + GraphSAGE detector	0.76	0.80	0.770 ± 0.036	0.808	89/107	59/67	23/30	–		
GraphSAGE, no ring scorer	0.88	0.72	0.780 ± 0.037	–	68/107	50/67	16/30	−0.010	[−0.044, +0.023]	0.664
GraphSAGE + scorer	0.84	0.70	0.755 ± 0.039	0.701	69/107	47/67	14/30	+0.015	[−0.021, +0.051]	0.547
Proposed (shipped)	0.76	0.72	0.718 ± 0.037	0.792	68/107	54/67	20/30	+0.052	[+0.023, +0.081]	0.003

TABLE VI

WHAT THE PAIRWISE DETECTOR'S ADVANTAGE DEPENDS ON: ISOLATION CONTROLS INSIDE THE FULL PIPELINE (SAME DETECTORS, TRACKER AND SCORER; 5 SCENARIOS × 10 OR 20 FRESH TEST SEEDS). Δ: SCENARIO-WEIGHTED MEAN PAIRED DIFFERENCE IN RING F1 (A MINUS B) WITH 95% CLUSTER BOOTSTRAP INTERVAL; p: STRATIFIED SIGN-FLIP TEST; (H): CONFIRMATORY COMPARISON (HOLM FAMILY OF FOUR); OTHERS EXPLORATORY. LAST COLUMN: RECALL OF HUB-HIDING RINGS, A / B.

A vs. B	Seeds	F1 A / B	Δ	95% CI	p	Δ hub_heavy	Hub-hiding
Keep hub links vs. cap 10, $1/\ln(1+d)$ (H)	2001–10	0.700 / 0.616	+0.084	[+0.048, +0.121]	0.002	+0.34	29/52 / 9/52
Keep hub links vs. cap 10, constant	2001–10	0.729 / 0.671	+0.057	[+0.039, +0.077]	0.007	+0.31	31/52 / 7/52
Hub cannot suffice vs. can, constant	2401–20	0.737 / 0.600	+0.137	[+0.106, +0.168]	<0.001	+0.44	72/104 / 9/104
Rare attribute cannot suffice, $1/\ln$ (H)	2401–20	0.729 / 0.729	0.000	[−0.028, +0.028]	0.980	−0.03	68/104 / 67/104
$1/\ln(1+d)$ vs. constant, no cap	2001–10	0.700 / 0.729	−0.028	[−0.068, +0.009]	0.218	0.00	29/52 / 31/52
$1/\ln$ vs. constant, both two-evidence	2401–20	0.729 / 0.737	−0.008	[−0.020, +0.003]	0.174	−0.03	68/104 / 72/104
Pairwise + cap 10 vs. rules + scorer	2001–10	0.616 / 0.611	+0.005	[−0.039, +0.051]	0.885	−0.30	9/52 / 27/52

ring leaves. The gain over the rules in Table III therefore has two sources: pairwise evidence with several detectors, which helps under noise, and keeping hub links, which helps under hub hiding.

Two exploratory follow-ups on further fresh seeds (protocol in `docs/research/round3_protocol.md`) sharpen this. First, the cost of a cap does not come only from attributes that could link a pair alone. A cap of 27 removes exactly the attributes that, at $\theta_e = 0.3$, can never link a pair without a second piece of evidence, and still costs -0.051 F1 (CI -0.067 to -0.035; cap 50: -0.034; cap 10: -0.087). Hub-hiding rings found fall from 64/111 to 39/111. Busy attributes therefore help as *supporting* evidence, added to a transfer or a rarer attribute. Second, the property that rare attributes link alone is what the adaptive rings need, but a step function with that property is not a substitute for the discount: $h(d) = 1$ for $d \leq 27$ and 0.2 above costs -0.041 (CI -0.061 to -0.021), mostly on `hub_heavy`, where the flat 0.2 lets busy attributes add up. A constant weight that requires two pieces of evidence ($\theta_e = 1.0$, the constant row of Table V) differs by 0.027 (CI -0.001 to 0.054), with fewer false rings, but finds 13/26 adaptive rings against 18/26 for $1/\ln(1+d)$.

D. A GNN-inclusive pipeline

The comparison above shows a learned node classifier ahead of the pipeline on some scenarios and behind it on others, and the hybrid shows the two are complementary. We therefore designed a GNN-inclusive pipeline under a separate protocol committed before any of its models was trained (`docs/research/evidgnn_protocol.md`), with four confirmatory hypotheses under Holm correction and fresh seed blocks. **EvidGNN** is a two-layer relation-aware GraphSAGE over three relations: user–attribute links, transfers, and the pipeline's evidence edges ($W \geq \theta_e$, top 16 per user). It has a *gate*: an attribute shared by more than 27 accounts, which cannot link a pair alone, passes no messages. It has 16 features, the GraphSAGE baseline's 12 plus four evidence statistics, and is trained exactly like the baseline. **EvidPipe** adds EvidGNN

TABLE VII

GNN-INCLUSIVE PIPELINE AGAINST EXISTING METHODS ON FRESH TEST SEEDS 4001–4020 (100 PAIRED STREAMS; SELECTIONS FIXED ON VALIDATION BEFORE THE RUN). F1 PER SCENARIO: ADVERSARIAL, BASELINE, DRIFT, HIGH NOISE, HUB HEAVY. Δ: EVIDPIPE MINUS ROW (SCENARIO-WEIGHTED, 95% CLUSTER BOOTSTRAP), p: STRATIFIED SIGN-FLIP. †STRONGEST EXISTING METHOD BY VALIDATION F1; THIS COMPARISON (H1) IS CONFIRMATORY, THE OTHERS EXPLORATORY.

Method	P	R	F1	PR AUC	Adv	Base	Drift	Noise	Hub	Δ	95% CI	p
EvidPipe: pipeline + EvidGNN + GNN-aware scorer	0.87	0.86	0.860	0.894	0.65	0.99	0.95	0.77	0.94	–	–	–
Pipeline + GraphSAGE detector (hybrid)	0.80	0.85	0.815	0.827	0.63	0.95	0.75	0.84	0.90	+0.046	[+0.021, +0.069]	0.003
Pipeline (shipped)	0.79	0.76	0.763	0.824	0.63	0.97	0.69	0.70	0.82	+0.097	[+0.073, +0.122]	<0.001
GraphSAGE, no ring scorer†	0.86	0.70	0.758	–	0.51	0.87	0.90	0.84	0.68	+0.102	[+0.067, +0.138]	<0.001
GraphSAGE + scorer	0.86	0.69	0.754	0.704	0.50	0.87	0.91	0.78	0.71	+0.106	[+0.073, +0.140]	<0.001
Rules + own scorer	0.90	0.58	0.679	0.705	0.46	0.97	0.71	0.42	0.83	+0.182	[+0.154, +0.208]	<0.001
EvidGNN alone (gated, evidence rings)	0.74	0.60	0.651	–	0.45	0.80	0.76	0.41	0.84	+0.209	[+0.180, +0.238]	<0.001

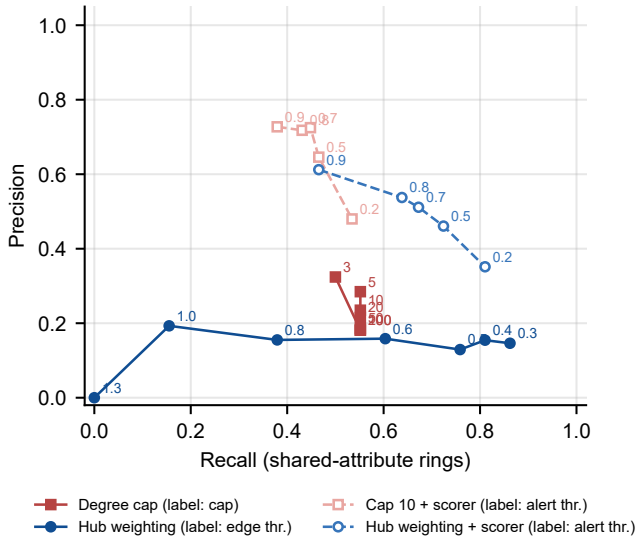


Fig. 4. Precision and recall on shared-attribute rings (micro-averaged over 4 scenarios × 5 test seeds). Solid: detector alone, swept over the degree cap or the edge threshold θ_e. Dashed: with the calibrated scorer, swept over the alert threshold θ_a (cap 10; $\theta_e = 0.3$). Best-F1 difference with the scorer (hub minus cap), each at the best alert threshold on these test runs (an oracle choice): 0.03, 95% bootstrap interval -0.03 to 0.10 (`results/e3b_frontier_ci`).

as a detector to the shipped pipeline (run on the whole graph every micro-batch) and replaces the ring scorer with one over the 19 ring features plus three statistics of the members' GNN probabilities (mean, minimum, fraction flagged). The scorer is trained on the training seeds with *out-of-fold* GNN scores: seeds 101–105 are scored by a GNN trained on 106–110 and vice versa. Node and alert thresholds were selected on the validation seeds, with the GraphSAGE baseline's grid widened first so that it did not end on its edge.

On fresh test seeds (Table VII) EvidPipe reaches F1 0.860 against 0.758 for GraphSAGE, the strongest existing method: a difference of +0.102 (95% CI +0.067 to +0.138, $p < 0.001$ after Holm correction, wins/ties/losses 61/17/22). It also beats the hybrid (+0.046, CI +0.021 to +0.069) and has the highest ring PR AUC. The gain is largest on `hub_heavy` (+0.267); on `high_noise` GraphSAGE remains ahead (−0.071). The ablations on a second fresh block replicate the gain over GraphSAGE (+0.088, CI +0.053 to +0.124). They also show that it does *not* come from what we designed the GNN to do. Switching the gate off *improves* EvidGNN (gate on minus off −0.063 F1, CI −0.084 to −0.043; the confirmatory test H2 is rejected). The evidence ring rule costs −0.040 for EvidGNN and −0.056 for GraphSAGE overall, while gaining +0.210 and +0.165 on `hub_heavy`. As a node classifier the gated model is weaker than GraphSAGE (−0.058 with the same ring rule). What matters is the scorer: with the shipped 19-feature scorer instead of the GNN-aware one, the same detectors fall from 0.843 to 0.742, below GraphSAGE alone (0.755). The two remaining hypotheses are not supported. With only 10% of the labels the gated model is not better than GraphSAGE (−0.022, CI −0.071 to +0.022). When a scenario is left out of training it loses more than GraphSAGE, not less (F1 drop 0.083 against 0.048). The rule-based pipeline's scorer, by contrast, loses nothing when a scenario is left out (0.001, CI -0.010 to 0.012), while GraphSAGE loses 0.057 (CI 0.037 to 0.078; seeds 3201–3220).

Because the gate did not help, we tested the simplest version, declared in a protocol amendment before any run, on a third fresh block (4801–4820): the shipped pipeline with the plain GraphSAGE baseline as a detector and the same GNN-aware ring scorer (**SagePipe**). It reaches F1 0.859 against 0.856 for EvidPipe. The two are equivalent within 0.05 F1 (difference +0.003, 95% CI −0.020 to +0.025). Both beat GraphSAGE (0.767; SagePipe +0.091, CI +0.064 to +0.119) and the hybrid (0.762; +0.097, CI +0.078 to +0.116). The hybrid uses the same detectors but a ring scorer without the GNN statistics, so these statistics account for the whole difference between the two.

The advantage holds only while the GNN's inputs look like its training data. In a generator variant where benign transfers are as large as ring transfers (median raised from 120 to 2,000; block 4701–4710; every model trained on the default generator), GraphSAGE alone finds no rings (F1 0.000). EvidPipe and SagePipe keep their recall but raise many false alarms (F1 0.450 and 0.399, precision 0.33 and 0.29). The rule-based pipeline (0.641) and the scored rules (0.661) degrade least. Every learned component here, including the pipeline's own scorer, uses transfer-size features. Without them

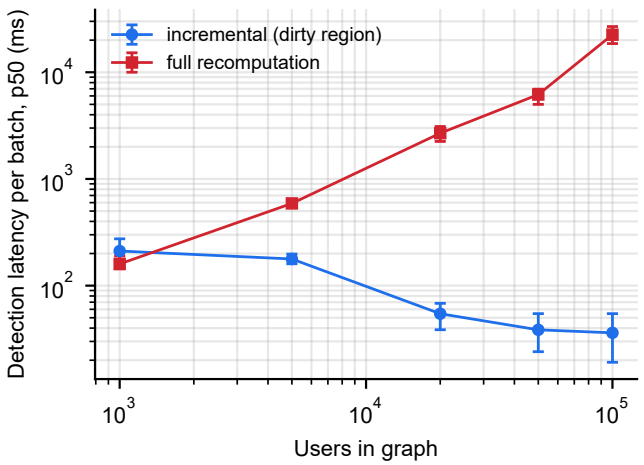


Fig. 5. Median per-batch detection latency as the graph grows (mean over 3 seeds; bars: per-seed minimum and maximum), incremental (dirty region) vs. full recomputation of the same detectors, in-memory store.

the shipped scorer's ring PR AUC on default data falls from 0.77 to 0.55 (seeds 3501–3520).

Hub size is the second shift. The largest attribute in the training snapshots is shared by 176 accounts (`scale_sweep`; at most 75 in the other scenarios). In the hub-size sweep (`hub_heavy` with an office hub of 500, 2,000 and 10,000 accounts, five seeds each, ranges; exploratory), EvidPipe raises 58 and 89 false rings per run at 500 and 2,000 accounts (F1 0.10–0.23). Its gated GNN gives the benign co-users of an oversized hub high probabilities, and the scorer trusts them for Louvain groups inside the hub. SagePipe does not have this failure: its median F1 is 1.00, 0.86 and 1.00 at the three sizes (ranges 0.92–1.00, 0.67–1.00 and 0.57–1.00), similar to the rule-based pipeline (0.86, 1.00, 0.86) and the hybrid. The GNN-inclusive pipelines run the GNN on the whole graph each micro-batch: at 10,000 accounts a stream takes a median 240 s against 11 s for the incremental pipeline.

The result is a GNN-inclusive pipeline that outperforms every existing method on this benchmark when test data resemble training data. Its simplest form, plain GraphSAGE plus the GNN-aware ring scorer, is as accurate as our gated design and more robust to hub size. The mechanism is the combination of rule-based candidate rings with a ring scorer that reads the GNN's member probabilities. Restricting the GNN's message passing around busy attributes does not help. This mirrors the degree-cap result: busy attributes are useful evidence for a learned model as well as for the pairwise rule.

E. RQ2: latency and scale

Table VIII and Fig. 5 grow the user population of the `scale_sweep` scenario at a fixed event rate (100 events/s) and compare incremental detection with recomputing the same detectors on the whole graph for every batch. Full recomputation grows at least linearly with the graph (faster between 50,000 and 100,000 users, where the 7.7 GB test machine was short of memory; we did not separate the two effects). The incremental region stays between a few hundred and two thousand users, because the fan-out limits stop the expansion at hubs; it even shrinks as the graph grows, since this scenario scales the number of devices, cards and IPs with the users (so an ordinary attribute is shared by fewer users) while hub sizes stay fixed. Its latency therefore stays flat: at 100,000 users a batch takes 36.1 ms incrementally against 22,501 ms in full. This comparison is a worst-case bound, not a deployment comparison: nobody would recompute the whole graph on every 50-event batch, the incremental figure (3 seeds) varies more than its mean, and the 50,000 and 100,000-user full figures are confounded by memory pressure. At 1,000 users the region already covers almost the whole graph, and the incremental mode is slower than full recomputation because of the cost of expanding the region. On the accuracy scenarios, incremental and full recomputation flag exactly the same rings in every run (ablation *no incremental* in Table IX), so restricting detection to the dirty region loses nothing there.

Against a timer. The realistic alternative is to run the same detectors on the whole graph on a timer. On the accuracy scenarios (30 to 200 users, 25 paired runs) a 1 s timer matches the incremental engine in F1 (0.69 vs. 0.71, $p = 0.624$) and time to detect (8.8 s vs. 9.5 s) at lower detection cost (22 vs. 37 ms per simulated second); a 5 s timer costs less still (6) but detects later (13.8 s), and a 30 s timer misses almost every ring, which ends before two passes confirm it (F1 0.01). On small graphs, incremental detection therefore buys nothing. The picture reverses with size. In `scale_sweep` with hubs that grow with the population (15% and 8% of users, other hubs up to 10%; 3 seeds), the cost per simulated second is 323, 464 and 173 ms for incremental detection at 1,000, 5,000 and 20,000 users, against 27, 371 and 4,170 ms for a 5 s timer (44,907 ms for recomputing every batch at 20,000). At 5,000 users the two overlap across seeds; at 20,000 users the timer needs more than one second of computation per simulated second in all three seeds, so it cannot keep up, while the incremental region stays near 912 users even though the hubs grow. On five fresh seeds at 10,000 users (revision container, run alone) the 5 s timer needs 1,148–1,359 ms per simulated second, more than one second in 5 of 5 seeds; the incremental engine is cheaper in 4 of 5 (median 185 ms), but in one seed it costs 1,270 ms, more than the timer. The incremental cost is not monotone in size (464 at 5,000, 173 at 20,000); a likely cause, which we did not isolate, is that the region is bounded by the fan-out limits while the number of touched users per batch falls as the graph grows at a fixed event rate. We did not run 50,000 users with a timer. Because these measurements come from different containers, we repeated the comparison in one environment, run alone, on five fresh seeds (3701–3705; ranges over seeds). Per simulated second, the incremental engine costs 223–349 ms at 1,000 users against 16–30 ms for the 5 s timer, so the timer is cheaper on small graphs. At 5,000 users the costs are 181–350 against 282–376, at 10,000 194–494 against 1103–1221, and at 20,000 83–192 against 4310–4413. From 5,000 users up the incremental engine is cheaper in every seed (5, 5 and 5 of 5), and the timer needs more

TABLE VIII

PER-BATCH DETECTION LATENCY (MEDIAN, MS; MEAN ± 95% CI OVER 3 SEEDS) AND USERS EXAMINED PER BATCH, INCREMENTAL VS FULL RECOMPUTATION, IN-MEMORY STORE. WITH THREE SEEDS THE t-INTERVAL CAN EXCEED THE MEAN; THE TEXT USES RANGES OVER SEEDS.

	Incremental		Full		
Users	Region	ms	Region	ms	Speed-up
1,000	976	211.0 ± 142.9	1015	159.7 ± 64.4	0.8×
5,000	1878	177.8 ± 46.7	5016	589.8 ± 67.3	3.3×
20,000	1324	54.6 ± 37.1	20010	2683.7 ± 1062.0	49.2×
50,000	1110	38.5 ± 38.2	50011	6181.6 ± 2576.4	160.4×
100,000	827	36.1 ± 44.2	100013	22500.5 ± 10278.5	623.0×

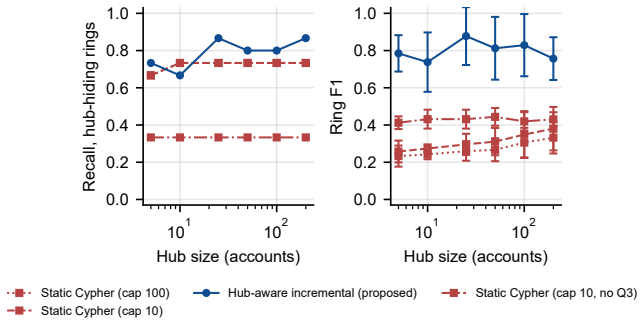


Fig. 6. A shared-IP ring hiding behind an office hub of growing size (`hub_heavy`, 5 test seeds per size). Left: recall of the hub-hiding rings; right: ring F1 (mean, 95% CI).

than one second per simulated second from 10,000 users. The static rules run on the same 5 s timer, which is why their time to detect is similar to ours on the accuracy scenarios (9.3 s vs. 9.5 s).

F. RQ3: robustness to evasion

Hub size. In Fig. 6 the office hub grows from 5 to 200 accounts. Recall of the hub-hiding rings does not fall with hub size for any method (proposed 71/90, static cap 10 65/90 over all sizes). The static rules do not find these rings through the shared hub: with the 3-cycle rule switched off, cap 10 finds 30/90, because the rings also move money among themselves in short cycles. What separates the methods is precision, and so F1 (0.80 vs. 0.31 for the unscored rules; 0.73 for the rules with their own scorer).

Hubs above the fan-out limit. Fig. 6 stops at 200 accounts, below the 250-user first-hop limit. We therefore resized the office hub to 500, 2,000 and 10,000 accounts (5 seeds each) and compared the shipped engine with full recomputation and with a "bound" rule that does cross an over-limit attribute, towards those co-members whose attribute weight with the touched user could reach the edge threshold. At every size the bound rule flagged the same rings as the 250-user limit, with identical precision, recall and region size (15/15 runs): it never added a user, so no pair in these streams shared two over-limit attributes. Full recomputation does not give identical runs. Tracing them showed two causes, neither of them the limit: a ring none of whose members is touched enters a region only partly, and greedy peeling of a connected component of several hundred users can return the ring on the whole graph but not on the region's part of it. In total it found no more hub-hiding rings (11/15 vs. 11/15 at 500, 9/15 vs. 10/15 at 2,000) and its F1 was not higher (0.79 vs. 0.79, 0.80 vs. 0.83), while the incremental region stayed at a median of 14 users against up to 2,076. At 10,000 accounts one full-recomputation run did not finish within 3.5 h on the test laptop, so there is no full-recomputation reference at that size. A constant weight (edge threshold 1.0, as selected in RQ5) found fewer hub-hiding rings incrementally than with full recomputation at 500 accounts (9/15 vs. 12/15; F1 0.74 vs. 0.85, lower in 3 of 5 runs; five seeds cannot reach significance), by the same two mechanisms; a single busy attribute cannot reach this threshold on its own. One full run of it took about 32 minutes at 2,000 accounts. So, for hubs up to 10,000 accounts, the 250-user limit itself hid no pair for either weight, but ring-level equality with full recomputation does not hold at these sizes. The static rules stay behind (F1 0.46, 0.60, 0.63 without a scorer; 0.65, 0.70, 0.74 with their own scorer).

Evasion strategies. Fig. 7 breaks recall down by the strategy each planted or random ring uses. The proposed method's advantage in recall comes mostly from rings without evasion (91/110 vs. 67/110 for cap 10) and rings with decoy edges (22/27 vs. 13/27). On hub-hiding rings the two are close (35/52 vs. 33/52), and identity rotation (4/14) and low-and-slow activity (3/13) defeat every method we tested, presumably because rotation removes the shared attributes the detectors rely on and slow rings accumulate little evidence inside one window.

Adaptive attacker. With feedback switched on, the attacker is told which of its rings were alerted and shifts its strategy mix. Over 10 test seeds it switched strategy 22 times and launched 22 adaptive rings, of which 18 were detected; without feedback, 9 of 11. We saw no change in the detection rate (difference 0.00, 95% interval −0.24 to +0.31), an interval too wide to rule out an effect, and the attacker's strategies are limited to the four we implemented.

G. RQ4: sensitivity

Raising the event rate from 10 to 100 events/s lowers the proposed method's F1 from 0.61 to 0.43 and raises its per-batch latency from 10.1 to 74.5 ms, since each batch touches more of the graph (Fig. 8). Larger planted rings are harder (F1 0.98 at size 3, 0.76 at size 8); a likely cause, which we did not isolate, is the 0.75 edge density the peeling requires. The

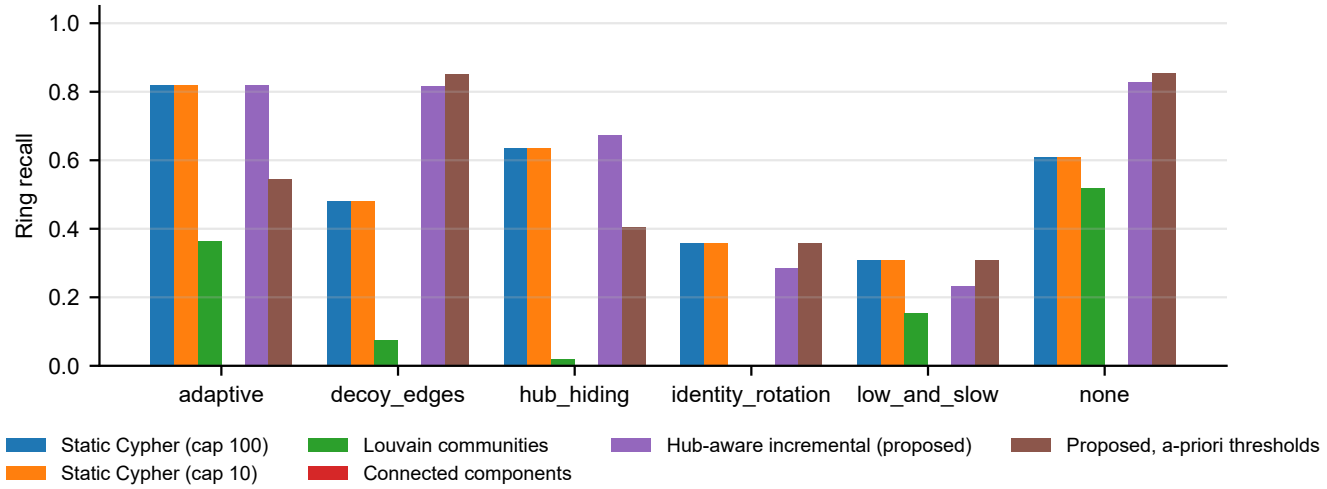


Fig. 7. Ring recall by evasion strategy, pooled over all runs of Table III (static rules without a scorer; Table V gives the recall by strategy for every method with its own scorer).

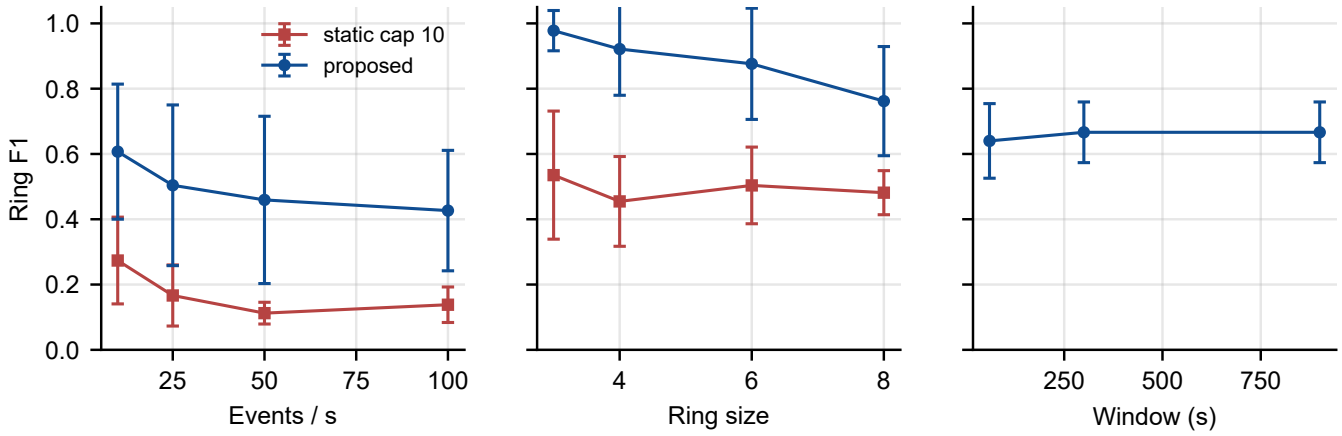


Fig. 8. Ring F1 of the proposed method and of the static rules (cap 10, without a scorer) when the event rate (`adversarial`), the planted ring size (`baseline`) and the view window (`drift`, proposed only) change; 5 test seeds per point. With their own scorer the rules reach F1 0.55 at the lowest and 0.40 at the highest event rate, and 0.97 at the smallest and 0.48 at the largest ring size.

view window between 60 and 900 s changes F1 little (0.64 vs. 0.67).

H. RQ5: ablations

Table IX removes one component at a time (20 paired runs). The calibrated scorer contributes most: without it F1 falls from 0.72 to 0.39 because precision drops to 0.28. Without the hub-aware detector, F1 falls to 0.36 and to 0.00 on `hub_heavy`, but precision rises to 0.98 at recall 0.25: most false candidates come from the hub-aware detector, and the scorer removes most of them. This suggests a separate alert threshold for rings the hub-aware detector found; selected on the validation seeds over a 6×6 grid, the best pair was the shipped single threshold (0.8, 0.8), so the refinement gains nothing here. Removing the cycle or the smurfing detector costs 0.64 and 0.65. Replacing the hub weight by a constant at the same edge threshold collapses `hub_heavy` (0.81 to 0.42) but helps `high_noise` (0.54 to 0.68). The effect depends on the scenario (heterogeneity $p = 0.005$), so the pooled test ($p = 0.161$) is not informative either way. That ablation keeps the edge threshold tuned for $1/\ln(1+d)$, which is unfair to the constant; the next paragraph gives each weight its own threshold.

Weight functions. We compared five weights $h(d)$, each with its edge threshold selected on the validation seeds and the alert threshold fixed at 0.8: the shipped $1/\ln(1+d)$ ($\theta_e = 0.3$), $1/d$ (0.1), $1/\sqrt{d}$ (0.2), a soft cap $1/(1+e^{(d-10)/2})$ (1) and a constant (1); for $1/d$ and the soft cap the selected value lies on the edge of this grid (widened in revision: $1/d$ moves to 0.05, the soft cap stays at 1.0, both now interior); for the constant, a wider grid in Section V-C selects 1.0 again as an interior optimum (1.2 is worse). On `hub_heavy` with 30

TABLE IX

ABLATIONS ON ADVERSARIAL, HUB_HEAVY, HIGH_NOISE, BASELINE ($\times$ 5 TEST SEEDS). ΔF1: PAIRED MEAN DIFFERENCE TO THE FULL SYSTEM; d_z: PAIRED EFFECT SIZE OF THE FULL SYSTEM OVER THE VARIANT; p: WILCOXON SIGNED-RANK, UNCORRECTED AND HOLM-CORRECTED OVER THE 7 TESTED COMPARISONS. FOUR SCENARIOS AND FIVE SEEDS, SO THE FULL SYSTEM'S F1 DIFFERS FROM TABLE III; THE HUB-WEIGHTING ROW AVERAGES EFFECTS OF OPPOSITE SIGN ACROSS SCENARIOS (SECTION V).

Variant	Precision	Recall	F1	ΔF1	d_z	p	p_{Holm}	Latency (ms)
Full system	0.75 $\pm$ 0.12	0.73 $\pm$ 0.12	0.72 $\pm$ 0.11	–	–	–	–	19.2
w/o hub weighting	0.65 $\pm$ 0.15	0.58 $\pm$ 0.14	0.60 $\pm$ 0.14	-0.12	0.42	0.161	0.462	24.0
w/o time decay	0.80 $\pm$ 0.11	0.72 $\pm$ 0.10	0.75 $\pm$ 0.09	+0.02	-0.18	0.277	0.462	17.6
w/o incremental scheduling	0.75 $\pm$ 0.12	0.73 $\pm$ 0.12	0.72 $\pm$ 0.11	0.00	–	–	–	18.4
w/o risk model	0.28 $\pm$ 0.09	0.82 $\pm$ 0.09	0.39 $\pm$ 0.10	-0.33	2.05	$<$0.001	$<$0.001	15.1
w/o hysteresis	0.72 $\pm$ 0.12	0.74 $\pm$ 0.10	0.70 $\pm$ 0.10	-0.02	0.37	0.154	0.462	22.7
w/o hub-aware detector	0.98 $\pm$ 0.05	0.25 $\pm$ 0.09	0.36 $\pm$ 0.12	-0.36	1.02	0.001	0.006	5.3
w/o cycle detector	0.75 $\pm$ 0.14	0.60 $\pm$ 0.11	0.64 $\pm$ 0.11	-0.09	1.02	0.002	0.009	15.4
w/o smurfing detector	0.72 $\pm$ 0.14	0.64 $\pm$ 0.15	0.65 $\pm$ 0.14	-0.07	0.93	0.003	0.013	19.9

paired seeds, the shipped weight reaches F1 0.79 and recall 0.87 of the rings hidden behind hubs; the constant reaches 0.81 and 0.83 ($p = 0.249$), $1/\sqrt{d}$ 0.79, while $1/d$ (0.56) and the soft cap (0.60, hidden-ring recall 0.27 at precision 0.99) are worse (p <0.001). Over the five scenarios the picture is the same: 0.73 for the shipped weight, 0.76 for the constant and 0.74 for $1/\sqrt{d}$, and 0.62 for $1/d$ and 0.68 for the soft cap. With the widened grid, $1/d$ at $\theta_e = 0.05$ reaches 0.69 over the five scenarios (-0.04 against the shipped weight, 95% CI -0.06 to -0.02) and 0.75 on `hub_heavy` (-0.05, CI -0.09 to +0.01), so most of its earlier deficit came from the grid edge. $1/\sqrt{d}$ is equivalent to the shipped weight within 0.05 F1 (90% interval -0.021 to +0.013); the constant is not significantly different but not shown equivalent (-0.063 to +0.011). Switching hub links off loses rings that share a hub (Section V-C); a strong discount costs little once the edge threshold is tuned for it, and the soft cap, which nearly switches busy attributes off, stays behind. Hysteresis makes no difference beyond 0.05 F1 (90% interval -0.001 to +0.040); time decay makes no significant difference but is not shown equivalent (-0.073 to +0.027), and without decay F1 is even slightly higher (0.75). The accuracy scenarios last 60–90 s, short against the 300 s half-life, so decay has little to act on here; its value on long streams is untested.

I. Public data

None of the public datasets we could obtain has device, card or IP links, so on all of them only the transaction detectors (temporal cycle, smurfing) run, and the results below say nothing about hub-aware weighting (Table X; sources and hashes in `docs/datasets.md`).

AMLworld [30] (HI-Small) is the only one with group labels: 370 laundering attempts of eight typologies. We replayed the first four days (2,284,182 transactions, 399,735 accounts) with a 1 h window and used the 55 attempts that lie entirely in that slice as ring ground truth. The detectors confirmed 303 groups; attempt recall is 0.00 and account-level precision and recall are 0.03 and 0.01. In other words, the transaction rules matched none of the attempts, and the groups they did confirm were almost all benign. Attempts span a median of 75 h, far longer than the window. A one-day window was too slow on our test machine (throughput fell steadily as the window filled), so attempts whose transfers are hours apart cannot be matched; and AMLworld's typologies (fan-in, scatter-gather, bipartite, stacks) are mostly not the relayed cycles and structuring our two transaction rules look for. The result is a negative one for short-window transaction rules on this data, not a comparison with published AMLworld models, which classify individual transactions.

Elliptic, as used in [26], [27], links each Bitcoin transaction to the later transactions spending its outputs, so its graph has no cycles, and amounts are not published. Neither transaction rule can fire: no account was flagged (0 of 203,769). We report it as not applicable to group detection rather than as a score.

AMLSim samples (IBM, 20,000 accounts, 200 planted cycles and/or fan-ins) were not flagged either (0, 0 and 0 accounts). The planted cycles are not time-ordered: none of the 21 we could reconstruct from equal hop amounts has its hops in time order (one cycle's five hops fall at steps 28, 113, 84, 59 and 78), whereas our cycle detector, by design, looks for money relayed around a loop.

VI. DEPLOYMENT, CALIBRATION AND EXPLANATION STUDIES

The experiments in this section were added after the main evaluation. Each has a protocol committed to the repository before its first result (`docs/research/`), and every amendment is recorded there with its reason.

A. RQ6: latency on the deployed stack

The latencies of Section V come from the in-memory store. We therefore replayed the paced scenarios of RQ1 and the users sweep through the deployed path: Redis Streams into the detection engine, with the graph in Neo4j 5.26. All 52 runs produced the same sequence of ring updates and the same flagged rings as the in-memory engine (52 of 52), so the store and the bus change timing, never results. Paced at simulated real time, a flag followed the event that completed its batch by a median of 176–438 ms across the five scenarios. Each detection reads its region through Cypher round trips and took 3 to 19 times as long as on the in-memory store for the same batches. When events arrive faster than the store can take

TABLE X

PUBLIC DATA, TRANSACTION DETECTORS ONLY (NO DEVICE/CARD/IP LINKS EXIST). ACCOUNT PRECISION/RECALL: ACCOUNTS IN CONFIRMED GROUPS VS. ACCOUNTS IN LAUNDERING TRANSACTIONS. ATTEMPT RECALL: SHARE OF LAUNDERING ATTEMPTS THAT LIE ENTIRELY IN THE SLICE AND ARE AT LEAST HALF COVERED BY ONE FLAGGED GROUP.

Dataset	Slice	Rows	Attempts	Acc. precision	Acc. recall	Attempt recall
amlsim_20K_cycle200	all	117,805	0	–	0.00	–
amlsim_20K_fanin200	all	118,250	0	–	0.00	–
amlsim_20K_fanin200cycle200	all	120,558	0	–	0.00	–
elliptic	all	234,355	0	–	0.00	–
amlworld	2022-09-01 to 09-05 (excl.)	2,284,182	55	0.03	0.01	0.00
attempt recall by typology: bipartite 0.00, cycle 0.00, fan-in 0.00, fan-out 0.00, gather-scatter 0.00, random 0.00, scatter-gather 0.00, stack 0.00						

them, Neo4j sets the ceiling: 35–52 events per second on the test laptop, with events queueing in Redis and flags arriving 9–19 s after their event. Varying the consumer's read size and blocking timeout, after HYADES [38], changed nothing measurable at these rates. Neo4j saturates before the consumer does.

B. RQ7: ring-level calibration and early warning

The scorer's isotonic calibration was fitted on snapshot rows, but analysts read scores of ring-level alerts. On 1383 ring-level units from the test seeds its expected calibration error is 0.10, and rings scored in the 0.8–0.9 and 0.9–1.0 bins were true rings 29% and 74% of the time. An isotonic calibrator [31] refitted on ring-level units of the validation seeds (301–303), a post-hoc recalibration of the kind Azbergen et al. [33] compare for AML risk scores, lowers the Brier score from 0.085 to 0.044. Ranking holds up better than calibration: precision at the top 5% of units is 0.79. Early warning, measured without look-ahead, is rare. Of 227 true rings with a forming phase, 30 (13%) were flagged before activation, a median of 2.2 s early, and 84% of first early warnings pointed at a real ring.

C. RQ8: explanation quality

We measured the explanations the product shows on the 262 alerts of the test streams. The unit is an alert (a ring at its peak calibrated score of at least 0.8); a model counterfactual is a proposed change of one actionable feature, and an alert can have none, one or several. 81 of the 262 alerts have at least one, 103 in all. Writing the unit test for counterfactual validity exposed a defect: counterfactuals aimed at the raw score of the calibrator's breakpoint, but the calibrator interpolates between breakpoints, so the proposed change left the calibrated score above the threshold. As shipped, 0 of the 103 counterfactuals flipped the decision; aimed at the interpolated crossing, 103 of 103 do. The correction was registered as a protocol amendment before the experiment ran. Any system that derives counterfactual targets from an isotonic calibrator can make the same mistake.

Coverage is the larger limitation: the other 69% of alerts have no single-feature counterfactual over the actionable features, because typical transfer size dominates their score. Under 5% noise in non-actionable features, after Artelt et al. [35], the contribution vector moves by a median of 0.026 (relative L_1) and the top three features never change for the median alert; for a linear model this is close to guaranteed, so it mainly checks the implementation. A scorer retrained on disjoint seeds, after Movin et al. [36], ranks the contributions similarly (median Spearman 0.92) and still raises 87% of the alerts, but it proposes the same first counterfactual for only 29% of them, so a counterfactual describes this model, not the ring. Against simulator ground truth, the members shown in the evidence subgraph of a correct alert are the true ring's (median precision 1.00, coverage 1.00). Because that subgraph is built from the flagged ring's members, this checks that the evidence shows the right accounts; it does not test whether the evidence explains the score. A deletion test checks whether the evidence explains the score: when each alert fires, we removed each group of the evidence in turn (member links to shared devices, cards or IPs, and transfers among members), recomputed the features and the risk, and compared the group whose removal lowers the risk most with the group behind the explanation's top-ranked feature. Over the 262 alerts they agree in 0.76 of cases (chance 0.35). That agreement is no better than always naming transfers: the explanation ranks a transfer feature first for 261 alerts, and transfers are the group with the largest effect in a share of 0.76. For the other 64 alerts the score depends more on shared devices or cards than the explanation's ranking suggests. Removing all listed evidence lowers the risk by a median of 0.90, removing as many unlisted links by 0.00; since the features are computed from the listed structure, that contrast checks the implementation rather than the explanation.

D. RQ9: a learned baseline and evasion strength

To test whether the rule stage is needed, we trained gradient boosting on the same 19 ring features over rule-free candidate groups (Louvain communities without a density filter). Training, calibration and threshold selection used the shipped scorer's protocol. On the test streams of RQ1 it reaches F1 0.555 against 0.731 (paired Wilcoxon $p < 0.001$, rank-biserial -0.85). Precision is not the problem: it is 0.94 against 0.76, with a third of the alerts. Recall is: 0.38 against 0.74. (Table V, on fresh seeds 2101–2120 with each method's own tuned scorer, reports the same baseline lower, at 0.427; the difference is the seed block and the threshold selection.) The gap is largest where hubs and evasion matter (`hub_heavy`: 0.40 against 0.79; `adversarial`: 0.24 against 0.59).

Turning up camouflage strength, after Liu et al. [4] and Hu et al. [8], separates the methods further. With up to eight decoy edges per ring action, the proposed method's recall of the decoy rings stays between 0.74 and 0.82, while Louvain falls to 0.00 and the learned baseline to 0.05. Slow rings remain the weakness found in RQ3: the proposed method recalls at most 0.25 of them, and static Cypher with a degree cap of ten recovers more (0.42–0.75), though at a precision near 0.04. In the 60 s scenario, strong slow and identity-rotation settings produce few rings, so those two sweeps have little statistical power. We report them as indicative.

VII. Threats to Validity

Simulator realism. All main results come from our simulator. Hub sizes, ring behaviour and evasion strategies follow our configuration, not observed fraud. A detector tuned to this generator may do worse on real traffic, and the absolute numbers should not be read as field performance. The same authors wrote the generator, its evasion strategies and the detector, so hub hiding is defined in a way the pairwise detector can, in principle, catch; the GraphSAGE baseline, which learns the generator's regularities from labels, is the better guide to what is learnable from these streams.

Synthetic labels. Ground truth is the set of rings the simulator planted. Benign groups that happen to look like rings count as false positives, and planted rings that never become visible in the stream still count against recall; both directions affect every method equally.

Learned components and generator labels. The GNNs and every ring scorer are trained on labels from the same generator they are tested on, so their advantage measures how well they learn this generator's regularities. The GNN-inclusive pipeline's gain was designed after the comparison with existing methods had been seen. We tested it only under a separate protocol on fresh blocks, and its simplest form (SagePipe) was added by a protocol amendment after the ablations had been seen. All learned parts use transfer-size features, and their advantage disappears when benign amounts shift (Section V-D). The GNN runs on the whole graph each micro-batch, so the GNN-inclusive pipeline does not inherit the engine's incremental cost.

Public data. The public datasets we could obtain carry no device, card or IP links, so they cannot test hub-aware weighting at all; on them only the transaction detectors run (Section V-I). Only AMLworld has group labels, and we could replay only a four-day slice with a one-hour window. Elliptic and the AMLSim samples turned out to be outside what the transaction rules can detect, for structural reasons we describe. A real-world test of hub-aware weighting would need data with shared-attribute links, which we did not find in public form with group labels. The IEEE-CIS fraud data (https://www.kaggle.com/c/ieee-fraud-detection) comes closest: e-commerce transactions with card, address and e-mail-domain fields and masked identity and device fields, labelled per transaction. It would let the shared-attribute detectors and the hub weight run on real links and measure false alarms around busy attributes, but without ring labels it cannot measure ring recall.

Tuning. Thresholds were selected on validation seeds of the same generator; the test seeds are independent draws but from the same distribution. The original test seeds (1001 and up) were reused across many experiments, some designed after earlier results on them were seen; every experiment added in revision therefore uses its own block of fresh seeds, declared before its first run. In the weight-function comparison the grid was widened until every selection was interior. The scorer was trained on the same generator and scenarios it is tested on; leaving one scenario out costs little (Section III), but a scenario type the generator cannot produce is not covered by that test.

Scale. The largest graph we measured has 100,000 users. A million users was not feasible on the 7.7 GB test machine: at 100,000 users less than 0.3 GB of memory was free and one full recomputation already took over 20 s per batch. In the scale scenario, attribute counts grow with the number of users while hub sizes stay fixed; graphs whose hubs grow with the population would give larger dirty regions. The 250-user first-hop limit was tested with hubs of up to 10,000 accounts, five seeds per size, on one scenario. It misses no pair that shares at most one over-limit attribute with the weights and thresholds we use, but ring-level equality with full recomputation holds only for small hubs: candidate extraction by greedy peeling is not local, so at 500 and 2,000 accounts individual rings differed in both directions.

Latency. The latencies of Section V come from the in-memory store on a laptop. Section VI repeats the measurement on Neo4j and Redis Streams. The detected rings are identical, but detection is several times slower and throughput is bounded by the store. Both figures describe one laptop running Docker, not a server. Experiments added in revision ran several at a time on a two-core container; we report no latency from them except the 10,000-user scale point, which ran alone.

Baselines. Table III uses the shipped scorer for the scored baselines; Table V gives every method its own. All baselines are our implementations or library calls configured to emit our ring object: FRAUDAR and Spade were designed for review and transaction graphs, and we reproduce Spade's output, not its latency. GraphSAGE is one architecture with fixed training settings and a simple node-to-ring rule; a temporal GNN, another grouping rule or more tuning could do better, and the pipeline is already behind it on F1. Node-level numbers are not comparable to published benchmarks.

Calibration. The shipped scorer is miscalibrated at the ring level (Section VI). Its scores rank alerts well but should not be read as probabilities until a ring-level calibrator replaces it. We did not re-run the main tables with the ring-level refit: it is a monotone map of the same scores, so with the alert threshold selected on the validation seeds it offers the same set of ranked operating points and changes only which one the grid picks, not the ranking behind Tables III and IX.

Attacker model. The adaptive attacker only shifts its mix among the implemented evasion strategies. We did not test a white-box attacker that exploits the detector's parameters:

rings smaller than 3 or larger than 25 members, sharing patterns whose edge density stays below 0.75, transfers kept under the cap $\beta_{\max}$ of the transaction term, or one distinct hub per pair of members. Each of these would bypass a fixed rule of our detector, so the evasion results are a lower bound on what an informed attacker achieves.

Explanations. The deletion test (Section VI) shows that the explanation's top-ranked evidence is no better than always naming transfers; the explanations should be read as descriptions of the model, not of the ring.

Reproducibility. With newer versions of NumPy, scikit-learn or NetworkX, the event streams are identical but detection results can change, so the reported numbers hold for the pinned versions in `requirements-lock.txt`, which the continuous-integration check uses.

Streaming load. "Streaming" here means micro-batches on a paced simulated stream. Accuracy falls as the event rate rises (F1 0.61 at 10 events/s, 0.43 at 100), and on Neo4j the store saturates at 35–52 events per second on the test laptop. Cheap engineering steps we did not test are reading each region with one batched query instead of several round trips, and caching attribute degrees between batches.

VIII. Conclusion

Keeping links through busy shared attributes helps a streaming ring detector only if no busy attribute can link two accounts on its own. A GNN added to the pipeline, with a ring scorer that reads its member probabilities, outperforms every existing method we tested by +0.102 F1 over GraphSAGE, on three fresh blocks. The gain comes from that scorer, not from restricting message passing around busy attributes, and it disappears when transfer amounts shift away from training, where the rule-based pipeline is more robust. Within the same pipeline, a degree cap costs 0.08 ring F1 and most hub-hiding rings, a weight that lets one attribute suffice costs 0.14, and two weights that both meet the condition are equivalent within 0.05 F1. Shape still matters at the margin: a step weight that meets the condition costs 0.041, and only weights that let rare attributes link alone find most adaptive rings. Against tuned rules, Louvain, Leiden and FRAUDAR-style peeling, each with its own calibrated scorer, the pipeline improves ring F1 on fresh test seeds, by 0.059 against the strongest of them (rules tuned per scenario, an oracle selection; exploratory). Without a GNN, a GraphSAGE classifier trained on ring labels from the same generator is ahead of the rule-based pipeline on one block (0.778 against 0.719) and level on another; it loses 0.057 F1 when a scenario is left out of training while the pipeline loses nothing, and it fails entirely when amounts shift. Restricting detection to the region a micro-batch touches misses no pair a full pass would link under a simple condition, but ring-level equality holds only for small hubs; the engine is no cheaper than a 5 s timer at 1,000 users, and cheaper than it in every seed from 5,000 users up when both run in one environment. Slow and identity-rotating rings are rarely found by any method, the explanations describe the model more than the ring, and all positive results are synthetic. On Neo4j and Redis Streams the detector flags exactly the same rings, at the cost of slower detection. The next steps are data with real shared-attribute links, GNN inputs that are robust to shifts in transfer amounts, exact incremental GNN inference, a ring-level and time-aware calibrator, evidence that persists across identity rotation, and a white-box attacker.

Data and code availability. All data are synthetic and generated by the code. The repository contains the simulator, scenarios, detectors, the trained scorers and GraphSAGE model, every result file (each row records the scenario hash, seed and a digest of the event stream; each run records the git commit and library versions), the protocols (committed to the repository before each run) with their amendments (`docs/research/`; the commit that first added each protocol is listed in `PROTOCOL_LOG.md`), and the scripts that regenerate every table and number in this paper from those files (`make tables`; `scripts/make_refine_tables.py`). `REPRODUCE.md` gives the commands; `requirements-lock.txt` pins the library versions with which the results reproduce exactly, and a test reruns a recorded run and compares it. Wall-clock latencies depend on the machine and are not reproducible by design. We will deposit the version used for this paper as a citable archive with a DOI at camera-ready.

References

[1] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based anomaly detection approaches," *Decision Support Systems*, vol. 133, p. 113303, 2020.

[2] P. Pan, "GB-FRD: Graph-based fraud ring detection in P2P payment networks," in *6th International Conference on Computer Vision and Data Mining (ICCVDM)*, 2025, pp. 457–462.

[3] H. Yin, Z. Zhang, Z. Wang, Y. Özyurt, W. Liang, W. Dong, Y. Zhao, and Y. Shan, "Behavioral graph fraud detection in e-commerce," in *IEEE International Conference on Data Mining Workshops (ICDMW)*, 2022, pp. 769–776.

[4] Z. Liu, J. Gao, H. Yu, and X. Luo, "A robust graph fraud detection model based on adversarial reweighting," *IEEE Transactions on Computational Social Systems*, vol. 12, no. 6, pp. 5213–5224, 2025.

[5] B. Zhang, H. Liu, and Y. Zhang, "Research on an incremental community detection algorithm for large-scale dynamic complex network," in *IEEE 3rd International Conference on Big Data Science and Engineering (ICBDSE)*, 2026.

[6] T. Abughofa, A. A. Harby, H. Isah, and F. Zulkernine, "Incremental community detection in distributed dynamic graph," in *IEEE 7th International Conference on Big Data Computing Service and Applications (BigDataService)*, 2021, pp. 50–59.

[7] B. Hooi, H. A. Song, A. Beutel, N. Shah, K. Shin, and C. Faloutsos, "FRAUDAR: Bounding graph fraud in the face of camouflage," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 895–904.

[8] P. Hu, Y. Qin, J. Wu, and Z. Yang, "HHGFD: Graph fraud detection through heterogeneity and heterophily characterization," in *IEEE 37th International Conference on Tools with Artificial Intelligence (ICTAI)*, 2025, pp. 818–825.

[9] J. Lv, Q. Du, Y. Zhang, Y. Han, and S. Li, "Multi-scale adaptive neighborhood awareness transformer for graph fraud detection," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 6076–6080.

[10] X. Li, X. Jiang, Z. Wang, C. Liu, Z. Ni, H. Wan, and X. Zhao, "Adaptive Gaussian mixture model with hierarchical propagation for one-class graph fraud detection," in *IEEE International Conference on Multimedia and Expo (ICME)*, 2025.

[11] B. Giles, B. Jeudy, C. Largeron, and D. Saboul, "Suspicious: A resilient semi-supervised framework for graph fraud detection," in *IEEE 35th International Conference on Tools with Artificial Intelligence (ICTAI)*, 2023, pp. 212–220.

[12] H. Peng, J. Zhao, L. Li, Y. Ren, and S. Zhao, "One-class adversarial fraud detection nets with class specific representations," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 6, pp. 3793–3803, 2023.

[13] H. Yu, W. Liu, N. Zhu, P. Li, and X. Luo, "IN-GFD: An interpretable graph fraud detection model for spam reviews," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 10, pp. 5325–5339, 2024.

[14] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[15] M. Charikar, "Greedy approximation algorithms for finding dense components in a graph," in *Approximation Algorithms for Combinatorial Optimization (APPROX)*, ser. Lecture Notes in Computer Science, 2000, pp. 84–95.

[16] W. Feng, S. Liu, D. Koutra, and X. Cheng, "Unified dense subgraph detection: Fast spectral theory based algorithms," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 3, pp. 1356–1370, 2024.

[17] A. Karaaslanli, P. A. Traganitis, and A. Konar, "Identifying adversarial attacks in crowdsourcing via dense subgraph detection," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.

[18] J. Jiang, Y. Li, B. He, B. Hooi, J. Chen, and J. K. Z. Kang, "Spade: A real-time fraud detection framework on evolving graphs," *Proceedings of the VLDB Endowment*, vol. 16, no. 3, pp. 461–469, 2022.

[19] A. Ahmad, L. Yuan, D. Yan, G. Guo, J. Chen, and C. Zhang, "Accelerating k-core decomposition by a GPU," in *IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 1818–1831.

[20] L. Li, H. Chen, P. Li, J. Han, G. Wang, and G. Zhang, "The k-core decomposition algorithm under the framework of GraphBLAS," in *IEEE High Performance Extreme Computing Conference (HPEC)*, 2021.

[21] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.

[22] V. A. Traag, L. Waltman, and N. J. van Eck, "From Louvain to Leiden: guaranteeing well-connected communities," *Scientific Reports*, vol. 9, p. 5233, 2019.

[23] L. A. Adamic and E. Adar, "Friends and neighbors on the Web," *Social Networks*, vol. 25, no. 3, pp. 211–230, 2003.

[24] D. Liben-Nowell and J. Kleinberg, "The link-prediction problem for social networks," *Journal of the American Society for Information Science and Technology*, vol. 58, no. 7, pp. 1019–1031, 2007.

[25] G. Papadakis, G. Koutrika, T. Palpanas, and W. Nejdl, "Meta-blocking: Taking entity resolution to the next level," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 8, pp. 1946–1960, 2014.

[26] S. Badjie, A. A. Susom, B. D. Sarker, N. bin Sharif, and S. A. Ashraf, "Ensembles or graph neural networks? Detecting illicit Bitcoin transactions under temporal drift," in *6th International Conference on Soft Computing for Security Applications (ICSCSA)*, 2026, pp. 557–561.

[27] R. N. Kumar and N. Oláh, "Temporal-aware graph attention network with XGBoost ensemble for blockchain transaction fraud detection," in *IEEE 4th Conference on Information Technology and Data Science (CITDS)*, 2026, pp. 331–334.

[28] B. Lu, T. Xiao, Y. Zhao, and L. Su, "LAGCN-AD: An LSH-attention-based graph convolutional network for blockchain anomaly detection," in *IEEE 6th International Conference on Electronic Technology, Communication and Information (ICETCI)*, 2026, pp. 131–135.

[29] S. B. Shah, "Advanced machine learning models for anti-money laundering (AML): Improving detection accuracy and efficiency," in *IEEE 1st International Conference on Secure IoT, Assured and Trusted Computing (SATC)*, 2025.

[30] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, "Realistic synthetic financial transactions for anti-money laundering models," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.

[31] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2002, pp. 694–699.

[32] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005, pp. 625–632.

[33] T. Azbergen, A. Tleubayeva, and A. Aituov, "Calibration-aware gradient boosting with temporal behavioral features for anti-money laundering detection," in *IEEE 6th International Conference on Smart Information Systems and Technologies (SIST)*, 2026.

[34] Z. Ding, X. Han, P. Liu, and M. Niethammer, "Local temperature scaling for probability calibration," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 6869–6879.

[35] A. Artelt, V. Vaquet, R. Velioglu, F. Hinder, J. Brinkrolf, M. Schilling, and B. Hammer, "Evaluating robustness of counterfactual explanations," in *IEEE Symposium Series on Computational Intelligence (SSCI)*, 2021.

[36] M. Movin, F. Siciliano, R. Ferreira, F. Silvestri, and G. Tolomei, "Consistent counterfactual explanations via anomaly control and data coherence," *IEEE Transactions on Artificial Intelligence*, vol. 6, no. 4, pp. 794–804, 2025.

[37] B. Meulemeester, R. M. B. de Oliveira, Y. He, and D. Martens, "Calculating and visualizing feature importance values for counterfactual explanations," *IEEE Access*, vol. 14, pp. 50 691–50 703, 2026.

[38] K. Bora, E. Kartsakli, and E. Quinõnes Moreno, "HYADES: Adaptive consumer tuning in Redis Streams via hybrid expert selection," in *IEEE 19th International Conference on Cloud Computing (CLOUD)*, 2026, pp. 518–528.